

Learning through tinkering

Designing interactive worked examples for
introductory-level data science students

Anna Fergusson
University of Auckland
New Zealand
a.fergusson@auckland.ac.nz

Sven Hüsing
Paderborn University
Germany
sven.huesing@uni-paderborn.de



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND



UNIVERSITÄT
PADERBORN

Research background

- International collaborative project involving **researchers from New Zealand and Germany** in the area of data science at the senior high school level
- Focus is **development of design principles** for tasks and computational environments
- Research involves teacher and student participants from high school and **introductory-level college data science courses**



Teaching context

[DATASCI 100 Data science for everyone](#)

Module 1: Producing data-based communications involving visualisations

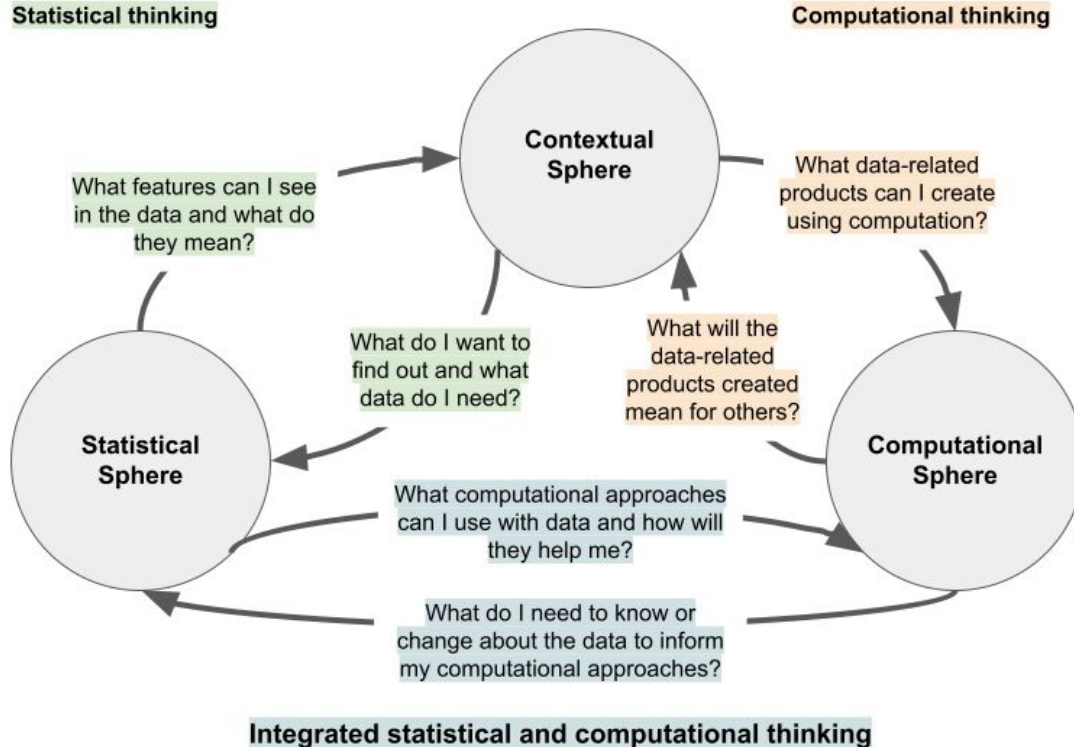
Module 2: Implementing algorithms for discovering and analysing features of data

Module 3: Creating data science products using informal generative models

Module 4: Developing models from data to make predictions and classifications

“... Across the course, students will be introduced to a wide range of data science applications and be supported to think creatively, ethically, and responsibly about how to use data, models, and technology to produce solutions to identified problems or challenges.”

Integrating statistical and computational thinking



Examples of data technologies covered in the course:

- R programming language
- APIs
- Google sheets
- Google Apps Scripts

Figure 8.4, p. 179, Fergusson (2022)

Course assessment overview

Group/individual participatory tasks

Interactive lectures
(no/low code)

Weekly labs
(interactive worked code examples)

Semester long project

Mid-term and final exam

The introductory-level data science education challenge

Many introductory-level students are programming novices

Students need **purposeful tasks** that engage them with data immediately rather than **learning syntax in isolation**

Lack of research specifically addressing the **pedagogical design** of introductory coding-based **data science** tasks

Specifically, how to effectively design tasks that **support students to integrate statistical and computational thinking** to learn from/with data, models, and technologies

Designing
interactive worked
examples for
introductory-level
data science
students

Epistemic programming

Epistemic programming (EP) denotes a programming practice taught not primarily as a professional practice, but as a skill that **empowers students to engage in personally or socially meaningful projects.**

Example: Exploring data using a **”tinkering” approach**, where students modify code, observe the resulting behaviour, and then make further code adjustments based on their reflections (Hüsing et al., 2023).

EP supports the integration of statistical and computational thinking - **intertwined process of knowledge acquisition and programming** - students still need scaffolds that introduce them to relevant domain-specific programming constructs.

Benefits and limitations of traditional worked examples

Worked examples are a common instructional resource for statistics and programming, and offer **context and guidance for data exploration and statistical modelling**, as well as provide a **code base that students can use**.

Interactive worked examples provide code that can be executed directly within the web browser (Fergusson, 2023), as well as other interactive aspects such as providing auto-marked quiz questions, or code exercises that can be auto-graded against an accepted solution.

Introductory data science students also need “worked examples” that encourage them to think creatively about data, stimulate their curiosity to use data, and assist their confidence to adapt the provided code for their own purposes.

Improving the design/use of worked examples

Fading out approach: Code aspects of the worked examples are **progressively removed and replaced with blanks** students need to complete, with decisions about what to “blank” based on an **analysis of the internal structure of the learning content** (Renkl et al., 2004).

Purposeful scaffolding: Use programming plans that provide the code needed to **produce a required output for a purpose**, with scaffolding that **directs learners' attention to the specific parts** of the code that could be adapted or extended (Cunningham et al., 2021).

Link to projects: Follow up worked example by having students **work on their own project** (Sweller and Cooper, 1985), which is similar in structure, but requires adapting the code so there are **opportunities for creativity** based on their individual interests.

Case study:

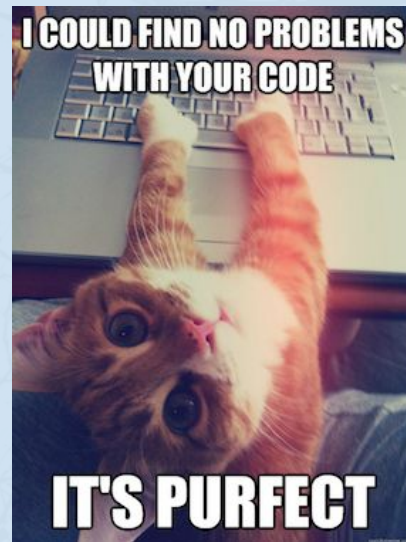
Exploring climate data from the NASA POWER API

Lab task 1A

Coding is for everyone!

Welcome to the first lab task for DATASCI 100!

For these labs, we are not focusing too much on the finer details of the code used. Rather than traditional exercises, where things might be defined first, we will instead use code to generate things for a purpose, and then review key features of the code after.



annafergusson.online/sdss2026

Localising the learning

In New Zealand, the [MetService](#) is the main organisation for collecting and sharing weather data. The MetService has a number of dedicated weather reporting stations throughout New Zealand and various offshore islands.

Auckland  MetService

No warnings in place

Today	2 May		19°	9°
Sun	3 May		19°	9°
Mon	4 May		19°	11°

Head to [their page which lists the towns and cities](#) and pick a location outside of Auckland.

Familiarising with the data source without code

NASA weather data

NASA (National Aeronautics and Space Administration) is a US-based government agency. They use satellites and associated models to provide free weather data for locations around the world through their POWER (Prediction Of Worldwide Energy Resources) service.

In the lectures, we learned how to visualise weather data about a location in New Zealand using NASA's data access viewer (power.larc.nasa.gov/data-access-viewer). Use the same steps as in the video to visualise (graph) the temperature data for the location within New Zealand you selected earlier in the lab.

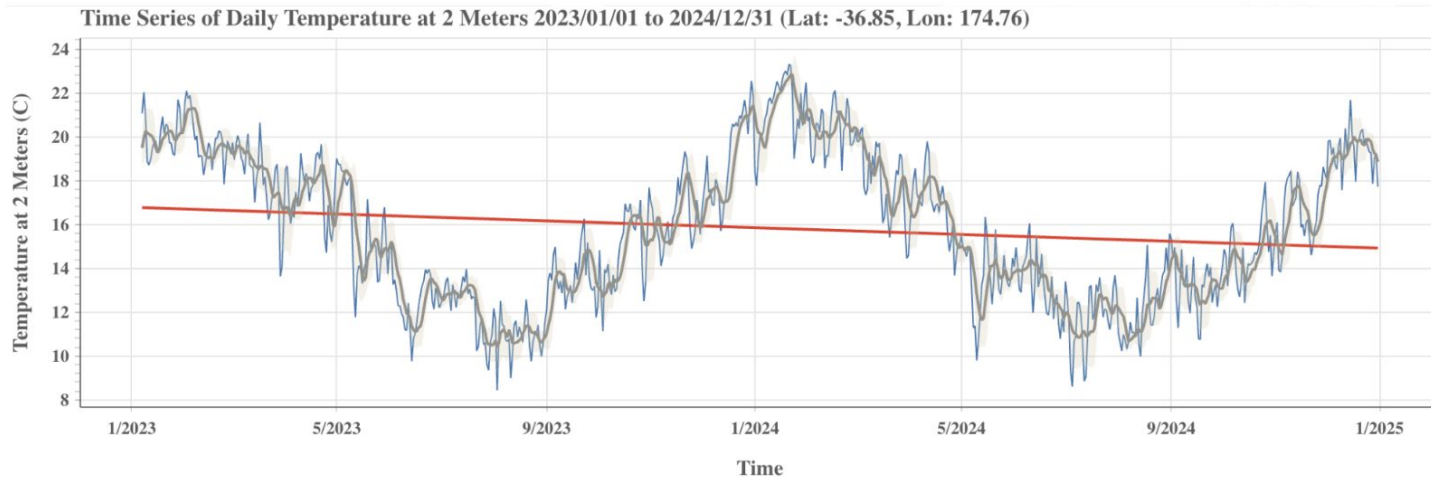
Note

We will use weather data sourced from NASA for the rest of this lab, including the final lab challenge.

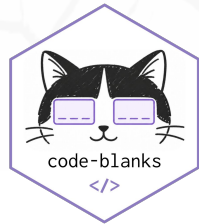
Recreating something familiar with code

How's the weather in Auckland?

The visualisation below was produced by the NASA data access viewer and displays the temperature in Auckland, along with some extra modelling components such as the red trend line (we won't worry about these additional modelling components for now).



Our version of “Hello world!”



We can recreate the visualisation above using R code.

Press the “Run code” button below to re-produce the visualisation.

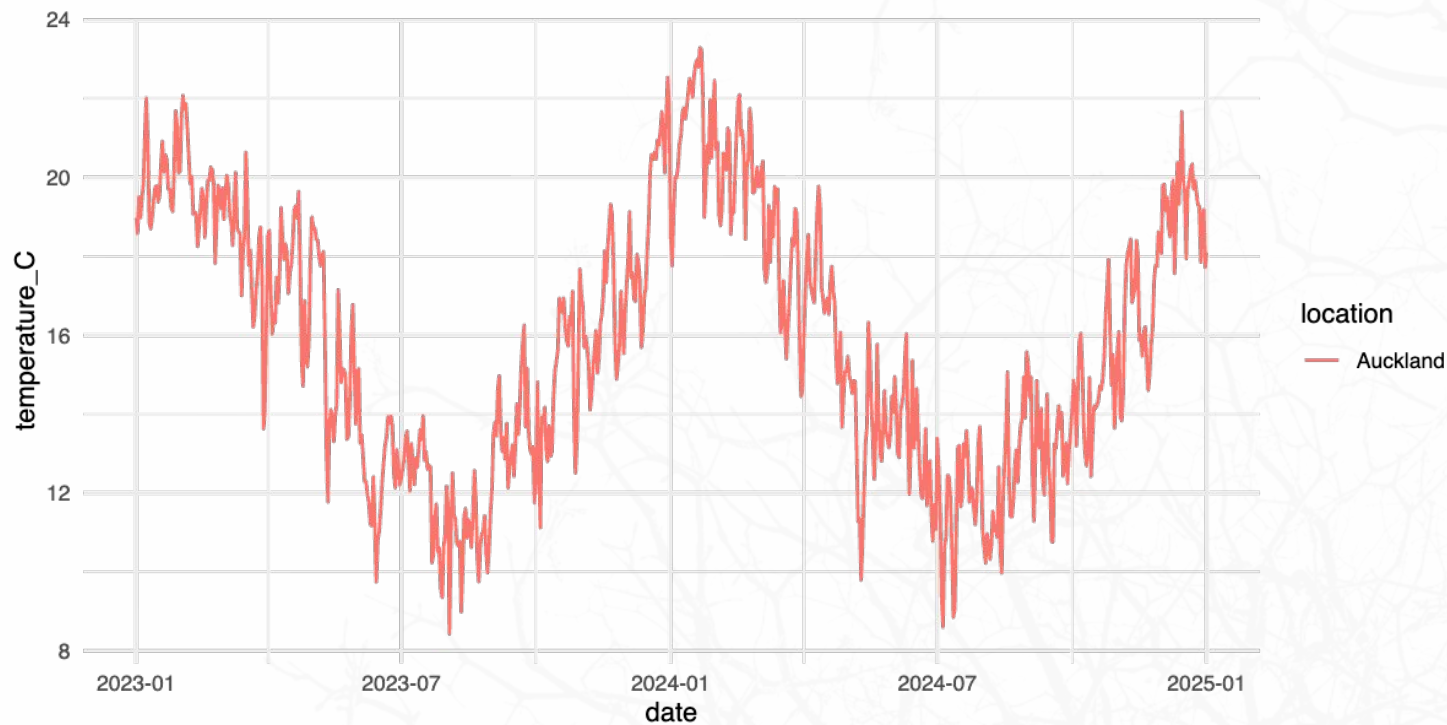
▶ Run Code

↺ Reset Code

📋 Copy Code

```
weather_data <- get_nasa_weather_data(location = "Auckland",  
                                       start_date = "2023-01-01",  
                                       end_date = "2025-01-01")  
  
ggplot(data = weather_data) +  
  geom_line(aes(x = date,  
                y = temperature_C,  
                colour = location))
```

Our version of “Hello world!”



Produce then reflect

Note

Code is just “fancy text” where certain features of the text can tell the computer what to do when reading/processing the code.

In the code we just “ran”, we used a **function** named `get_nasa_weather_data` to do just that - get weather data from NASA!

To get weather data, the computer needs to know where you want the data from, which is why we wrote `"Auckland"` after the `location =` part of the code within the `()` brackets.

Tinkering with the programming plan

- Give an instruction to run code and see what happens
- Provide some explanation or motivation to tinker with the code
- Make changes to the code and reflect on what is produced
- Ask students to write down their ideas or respond to specific questions

Use the same programming plan (code example) to introduce and try out new computational knowledge.

At the same time, we try to keep integrating statistical thinking as well!

Examples of tinkering-based micro-cycles

Don't misquote me!

Because Auckland is the name of an actual place, we needed to put `"` quotes around it.

Run the code below **just as it is without changing it** to see what happens if we don't use quotes!

[▶ Run Code](#)[↺ Reset Code](#)[📄 Copy Code](#)

```
weather_data <- get_nasa_weather_data(location = Auckland ,  
                                     start_date = "2023-01-01",  
                                     end_date = "2025-01-01")  
  
ggplot(data = weather_data) +  
  geom_line(aes(x = date,  
                y = temperature_C,  
                colour = location))
```

Error:
object 'Auckland' not found

Examples of tinkering-based micro-cycles

We can use R code to get data about more than one location at the same time.

Run the code below to see how the temperature data compares between Auckland and Christchurch.

▶ Run Code

↺ Reset Code

📋 Copy Code

```
weather_data <- get_nasa_weather_data(location = c("Auckland", "Christchurch"),  
                                     start_date = "2023-01-01",  
                                     end_date = "2025-01-01")  
  
ggplot(data = weather_data) +  
  geom_line(aes(x = date,  
                y = temperature_C,  
                colour = location))
```

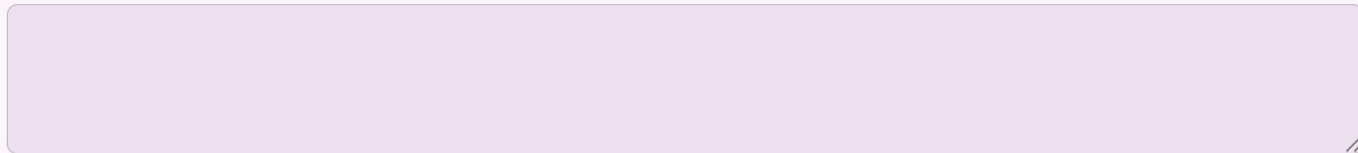
Note

The `c()` function was used here to supply more than one location.

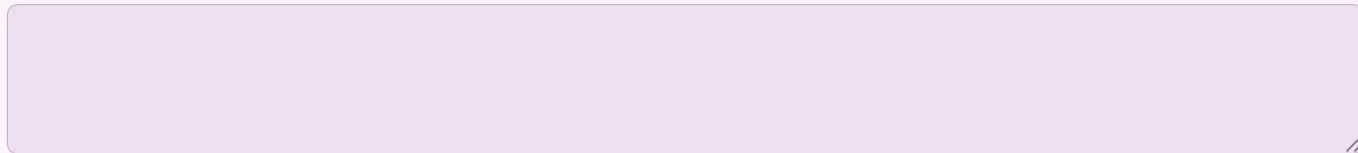
Examples of tinkering-based micro-cycles

Try changing the code above to answer the questions that follow.

Is Auckland generally warmer than Christchurch?



Do the temperatures in New York vary (change) less than they do in Auckland?



Tinker questions

Now adapt the code as needed to answer the following questions.

Click on the statements that are TRUE

Generally, Auckland is warmer than Christchurch.

Samoa is generally a lot hotter than Auckland, and the temperatures do not vary as much.

The general pattern for the temperatures in Melbourne across the year is very different from that for Auckland.

The temperatures in New York do not vary (change) as much as they do in Auckland.

The final lab challenge - personalised worked examples

For the DATASCI 100 lab tasks (interactive worked examples), each task ends by students completing a lab challenge.

This provides an opportunity for students to re-use and adapt the code and ideas covered in the lab, based on their own choices or interests.

Completing the final lab challenge also provides the students with their own worked example to use for reference for later work, including the semester long project.

Choose your own locations

There are two more variables that are available using the `get_nasa_weather_data()` function:

- `wind_speed_mps` (the windspeed in miles per second)
- `humidity_percent` (the humidity as a percentage)

Here's what you need to do for the final lab challenge.

- select either wind speed or humidity to explore as your variable of interest
- select two different locations in the world where you think the wind speed or humidity data will show different patterns over time when compared (do not include Auckland as one of the locations)

Complete the code blanks

Adapt and run the code below so that:

- the two locations used for the visualisation matches your selection above
- the last three years of data is used
- the variable used for the visualisation matches your selection (wind speed or humidity)
- the labels inform the reader about the locations, variable of interest, time period, and source of the data used for the visualisation

▶ Run Code

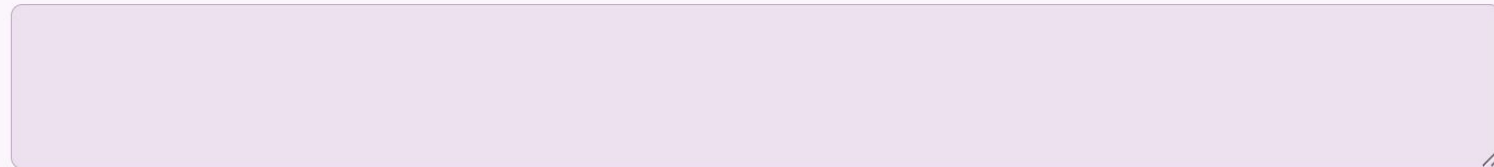
↺ Reset Code

📄 Copy Code

```
weather_data <- get_nasa_weather_data(location = "Auckland",  
                                     start_date = ,  
                                     end_date = )  
  
weather_data %>%  
  ggplot() +  
  geom_line(aes(x = date,  
                y = temperature_C,  
                colour = location)) +  
  labs(title = ,  
        subtitle = ,  
        caption = ) +  
  scale_x_date(date_breaks = "3 months",  
               date_labels = "%d-%b\n%Y",  
               date_minor_breaks = "1 month")
```

Reflect on your learning

Briefly describe in the box below what you think the visualisation reveals/communicates about how your variable of interest compares between the two locations over the last three years.

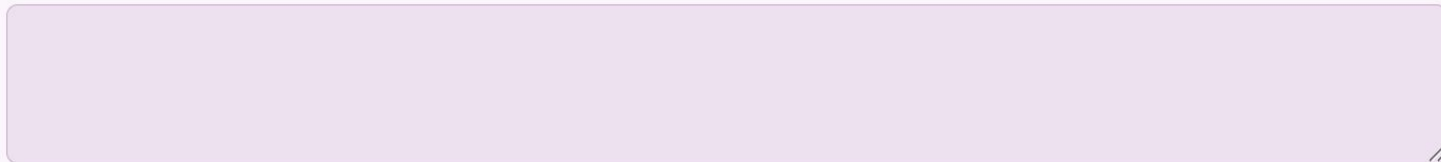


Briefly describe in the box below one thing that you thought was interesting statistically about what you explored in this lab.

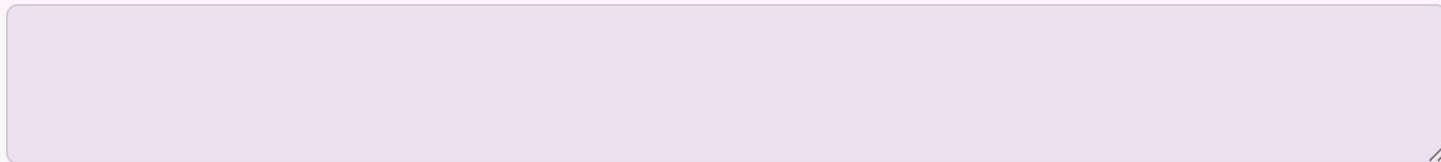


Reflect on your learning

Briefly describe in the box below one thing that you thought was interesting computationally about what you explored in this lab.

A large, empty, light purple rectangular box with rounded corners, intended for a user to write their reflection. A small pencil icon is visible in the bottom right corner of the box.

Think back across both the lectures and the lab task, and identify something that sparked your curiosity, and made you think "I wonder ..."? Write this "I wonder" question below!

A large, empty, light purple rectangular box with rounded corners, intended for a user to write their reflection. A small pencil icon is visible in the bottom right corner of the box.

Towards design principles for interactive worked examples (IWEs)

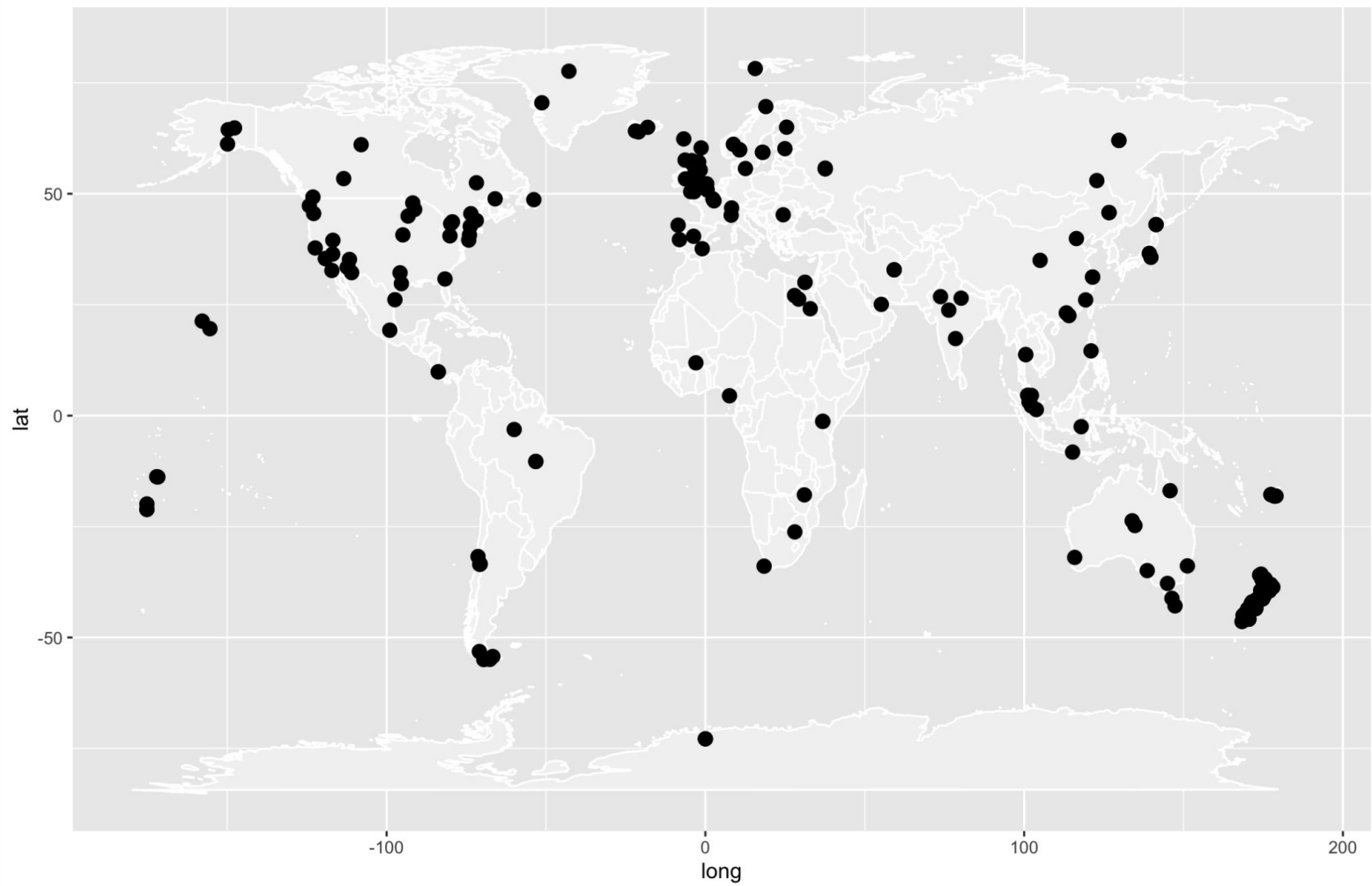
Overarching task design goals

To **frame the learning of computer programming** for introductory-level data science students **in an epistemic sense**, that is,

to present coding as a way of **knowing and making sense of the world through data**, rather than just learning syntax and rules.

To **design** our interactive worked examples in a way that:

- supports **all students to participate**, promotes a **tinkering mindset**,
- encourages **curiosity and creativity** about the world, and,
- **values diverse perspectives**.



Our four design principles

The interactive worked examples should:

- present a **cohesive and carefully sequenced collection of programming plans**, which can be combined to solve a particular problem.
- use **"code blanks" that indicate parts of the code to be changed** when using it **for one's own goals**.
- use **"tinker questions" or instructions** that guide students to complete or **change the code blanks** and then **reflect on what is produced**.
- be followed up by students **working on their own project**, which is **similar in structure**, but **based on their individual interests**.

Example of principles applied within Jupyter notebook

1 Reading in the data

Download data records for a specific location and time period

To download data records for the four attributes mentioned above for any location and save them to a DataFrame, we use the method `get_data_for`. The following parameters are used:

- 1st parameter: Name of the location or country (mandatory)
- 2nd and 3rd parameters: Time period from which the data should be taken. Format: `YYYYMMDD` in quotation marks, e.g. for 21 April 2024: `'20240421'`
 - If no second or third parameter is specified, the time period is automatically set to the current year (1 January to today).

Note: The data is often not published on a daily basis, meaning that the data for today, yesterday or the day before yesterday may not yet be included in the dataset.

Enter any town below in the `get_data_for` call to retrieve the data for that town for 1 March 2026. The data is displayed directly using the display command.

```
df_wetter = get_data_for("  ", "20260214", "20260214")
```

```
display(df_wetter)
```

Run Code

Copy Code

Some student reflections after the first lab (IWE)

“I found it interesting that there was a clear difference in humidity patterns between Guangzhou and Moscow. **Guangzhou stays humid throughout the year, while Moscow has much more seasonal variation**, especially drier winters. **I used to live in China, so this made a lot of sense. I still remember how constantly humid it felt**, especially in the south. I’ve studied weather-related data before in past courses, but this exercise helped reinforce that knowledge and **reminded me how useful it is to connect data with real-life experiences.**”

“When I did the Final Lab Challenge above I originally **left the start and end date fields blank to see what would happen**, and I thought it was kind of interesting that **something in the background selected a year-long time period for me**. I would have assumed that with no time period selected, I would have created a graph of the entire time period we had data for, but this wasn't the case.”

“**I really enjoyed using R** and coding to make the data visualisations **instead of just clicking some buttons**. I found it interesting that **we could write what we want to see and then make it happen.**”

“With **chatgpt** so readily available, **any code I've been stuck on I tend to resort to that** through the rest of my degree. But the **coding knowledge required** in these labs was **more of an understanding than an ability to straight write code**, which I think meant that **I engaged with it much more** than I have in many other courses.”

Student reflection from DATASCI 100

Future research work

Use data collected from student interactions with DATASCI 100 lab tasks to evaluate the cognitive demands of using “code blanks”

Identify and describe the different pedagogical uses of the code blanks within the DATASCI 100 lab tasks

Use student eye-tracking, digital interaction, and interview data to investigate how/if German high school students found the interactive worked examples supportive in their individual programming learning processes

Share examples of Jupyter notebook implementations, as well as Quarto generated webpage implementations (e.g. [{code-blanks} Quarto extension](#))

Related work

Development of more Epistemic Programming tasks and ressources:
Building an interdisciplinary community around the idea of Epistemic Programming
in the context of an interest group

=> Online Sessions, in which we will discuss and share approaches, instructional
material and experiences with Epistemic Programming

If you want to become part of the interest group, you can sign up here:

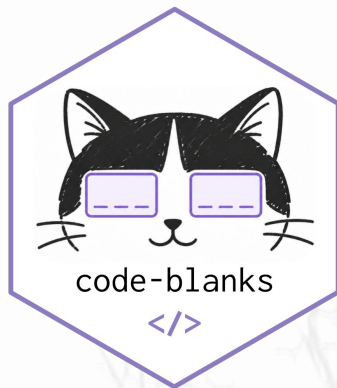
go.upb.de/EP-interestgroup



Links & resources



bit.ly/tinker-code



- PDF of the talk slides
- Extended abstract for this talk (includes references)
- Demo interactive worked example (DATASCI 100 Lab task 1A)
- {code-blanks} Quarto extension

Learning through tinkering: Designing interactive worked examples for introductory-level data science students

Anna Fergusson

University of Auckland

a.fergusson@auckland.ac.nz

Sven Hüsing

Paderborn University

sven.huesing@uni-paderborn.de
